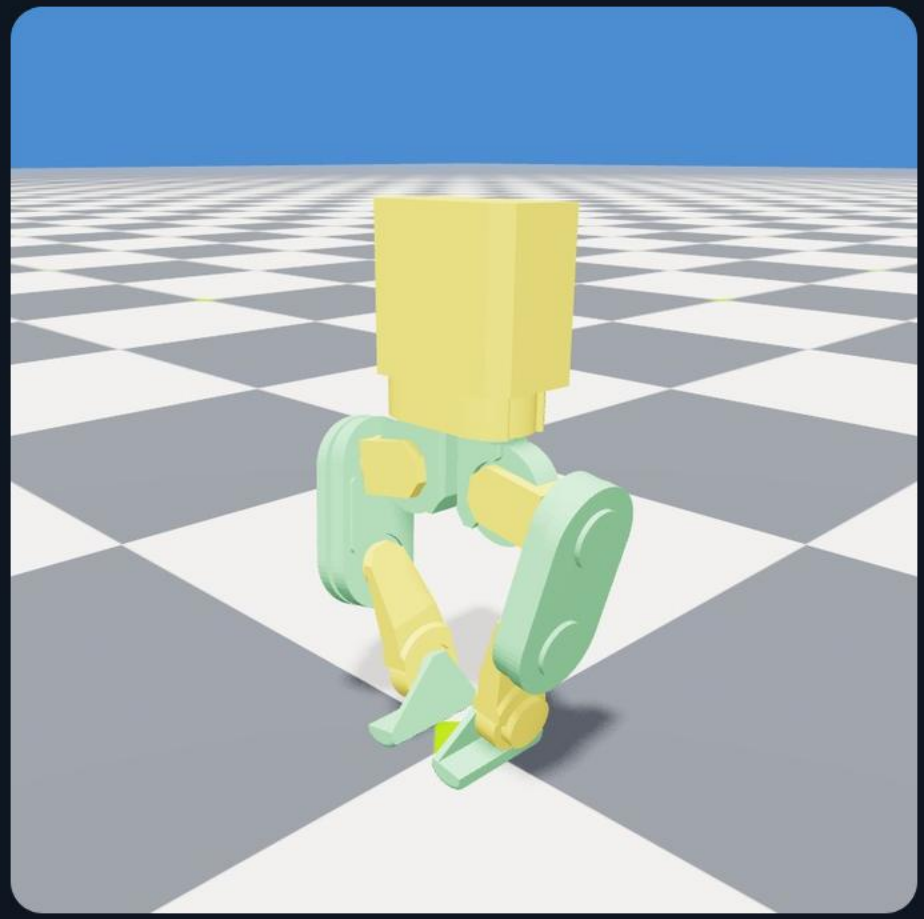
ENGINEERING WORKSHOP · ISAAC LAB 3 + NEWTON

Simulation for Humanoid

Week 4 — Reinforcement Learning & your own humanoid project

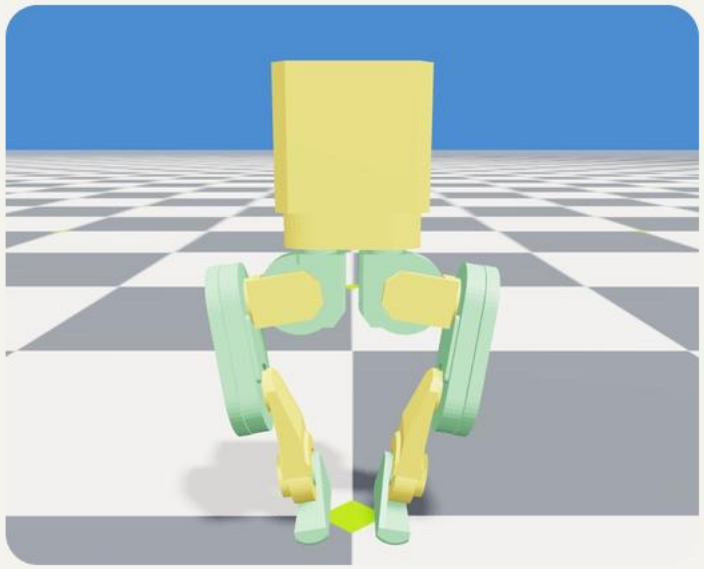
Isaac Lab 3.0 + Newton 2026 edition

Pannatorn Bongkotamphon
Dr. Sukhum Sattaratnamai



WEEK 4

Agenda



01 Recap + เป้าหมายวันนี้

02 Glimpse of Reinforcement Learning

03 สร้างโปรเจกต์หุ่น poclegs เอง

04 Lab 4.1–4.3: init pose, zero agent, train

05 Exercises

RECAP

ที่ผ่านมาใช้หุ่นที่ Isaac Lab เตรียมไว้ให้

Isaac-Cartpole

train → play

คำสั่ง isaaclab train / play กับ Newton viewer

Spot · Go2

locomotion task

reward, terrain, Hydra override, params/env.yaml

G1 · H1

humanoid

pretrained policy, ขับด้วยคีย์บอร์ด, ONNX

วันนี้: เอาหุ่นของเราเอง (ไฟล์ USD + config) มาสร้างเป็น task ใหม่ แล้วเทรนให้เดิน

MEET OUR ROBOT

poclegs

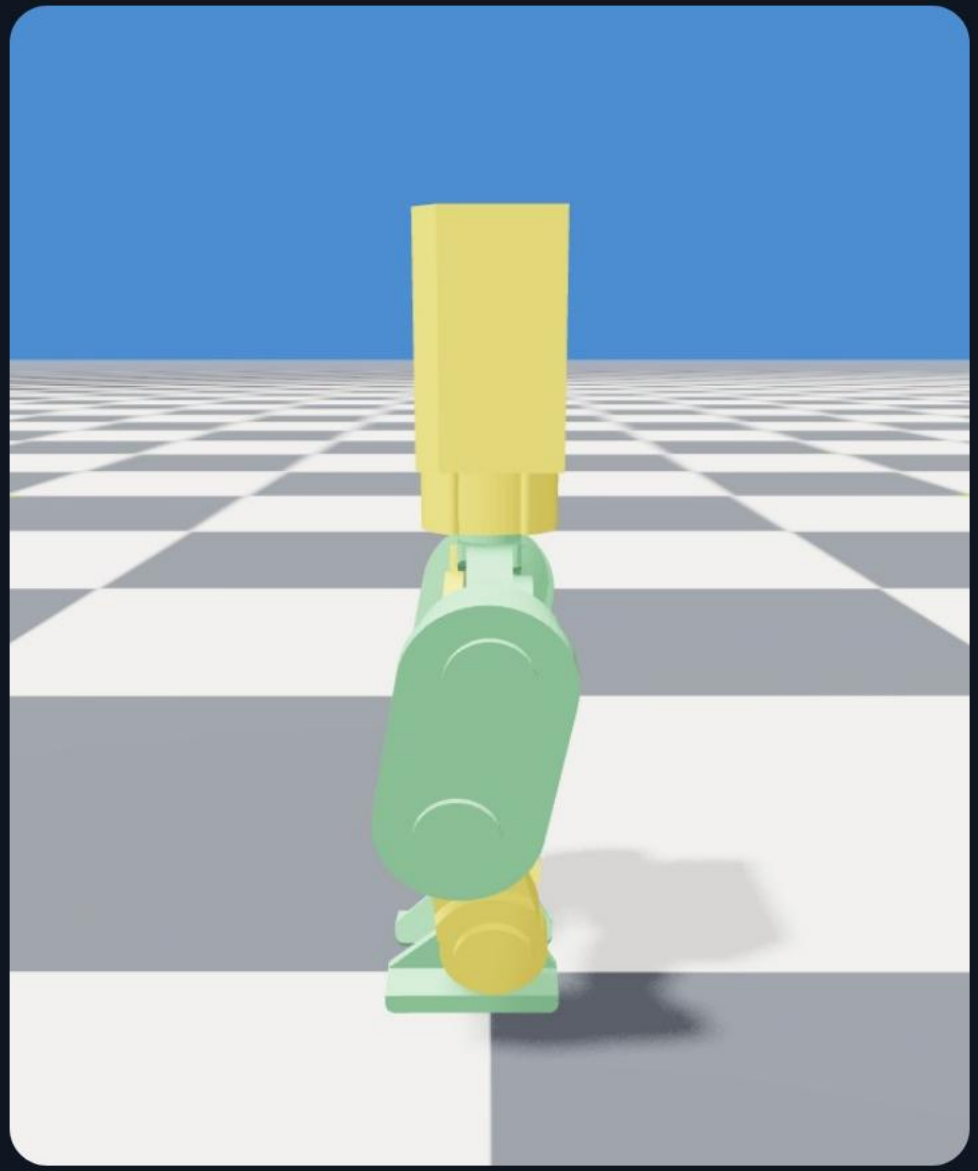
หุ่นสองขาตัวเล็กจาก repo BipedalRobotSim (Skyentific) ใช้ไฟล์ supplementary เดียวกับต้นฉบับ

10
ข้อต่อ (ขาละ 5)

17.5 kg
ลำตัว 4.5 + ขา 2 × 6.5

0.45 m
ความสูงลำตัวตอนยืน

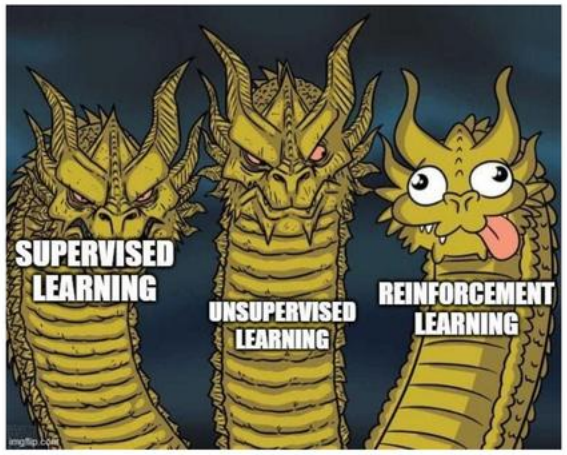
R-A-F
ลำดับสะโพก HR → HAA → HFE



LECTURE

Glimpse of Reinforcement Learning

ลองทำเองแล้วเรียนรู้ ไม่มีใครเฉลยให้



MACHINE LEARNING OVERVIEW

ML สามแบบ ต่างกันที่ “สัญญาณสอน”

Supervised

ได้ตัวอย่าง + เฉลย

ลด error ระหว่างคำทำนายกับ label

Unsupervised

ได้ข้อมูลไม่มี label

หา pattern / จัดกลุ่มเอง

Reinforcement

ได้แค่ “คะแนน” (reward)

ลองผิดลองถูก ให้ได้คะแนนรวมมากที่สุด

เขียนเฉลยท่าเดินทุกข้อต่อไม่ได้ แต่ให้คะแนนได้ว่า “เดินตามคำสั่ง ไม่ล้ม” ดีแค่ไหน เลยใช้ RL

VOCABULARY

ศัพท์ 5 คำ เทียบกับ poclegs

คำ	ความหมาย	poclegs
Agent	ตัว AI ที่ตัดสินใจ	neural network (policy)
Environment	โลกรอบ ๆ รวมตัวหุ่นด้วย	พื้น + ฟิสิกส์ Newton + ตัวหุ่น
Observation	สิ่งที่ agent รับรู้ได้	ความเร็ว, แรงโน้มถ่วง, มุมข้อต่อ, คำสั่ง (42 ค่า)
Action	สิ่งที่ agent ทำได้	มุมเป้าหมายของ 10 ข้อต่อ
Reward	คะแนนว่าตอนนี้ดีแค่ไหน	เดินตามคำสั่ง (+), เท้าลื่น/ตัวล้ม (–)

```
observation → agent → action → environment → observation', reward → …
```

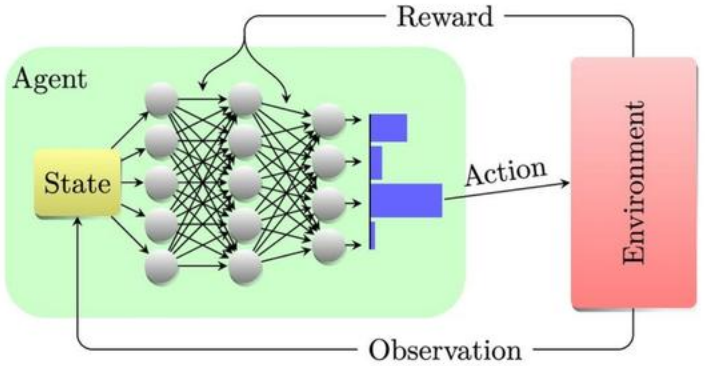
POLICY π

policy = ฟังก์ชันเลือก action

```
π(s) = a    deterministic
π(a | s)    probability (PPO)
```

RL คือหา policy ที่ดีที่สุด โดยใช้ **MLP** (multi-layer perceptron) เป็น policy

`42 obs` → `512 · 256 · 128` → `10 actions`



REWARD VS RETURN

reward ต่อ step, return ต่อ episode

Reward r

คะแนนที่คิดทุก timestep (50 ครั้ง/วินาที)

\+ เดินได้ความเร็วตามคำสั่ง

– เท้าลื่น, ตัวเอียง, ใช้แรงมาก, สั่งกระตุก

Return G

ผลรวม reward (ลดน้ำหนักด้วย γ = 0.99) จนจบ episode

time_out ครบ 20 s ตัดจบ (ไม่ได้ทำพลาด)

termination ลำตัวแตะพื้น ถือว่าล้ม

ใน log การเทรน: Episode_Reward/<ชื่อ term> และ Episode_Termination/<เหตุผล>

HOW IT LEARNS

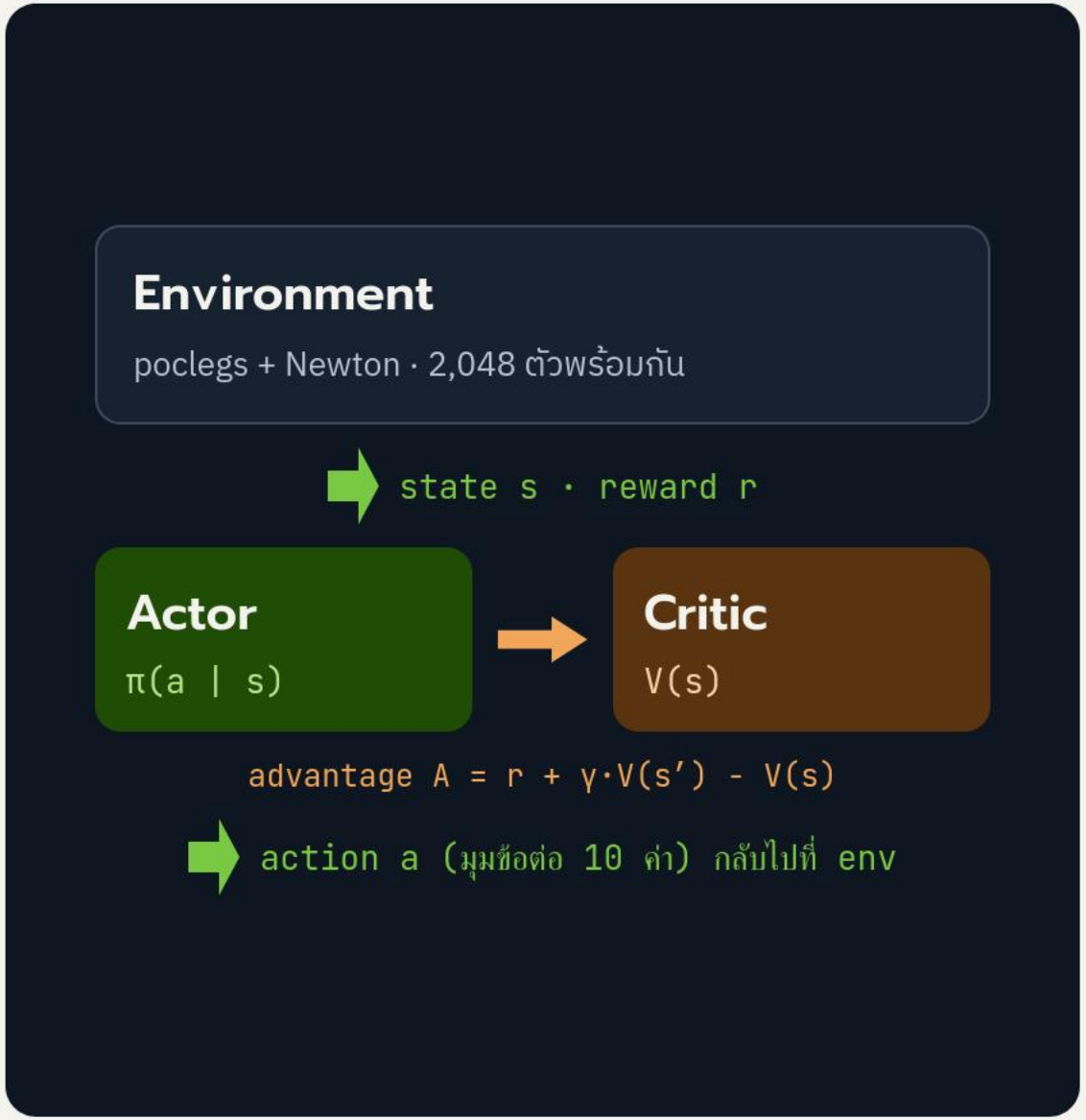
Actor + Critic

1 **Critic** (MLP อีกตัว) เดาว่า state นี้จะได้ return เท่าไร

2 เทียบกับที่ได้จริง → **advantage** “ดีกว่าที่คาดไหม”

3 **Actor** (policy) ปรับให้ทำ action ที่ advantage เป็นบวกบ่อยขึ้น

PPO · GAE (λ = 0.95) · TD error · rsl_rl



MARKOV DECISION PROCESS

อนาคตขึ้นกับ “ตอนนี้” เท่านั้น



Markov

ฟิสิกส์: รู้ตำแหน่ง + ความเร็ว ก็ทำนายต่อได้ ไม่ต้องจำอดีต

Non-Markov

จั่วไพ่: โอกาสเปลี่ยนตามไพ่ที่ออกไปแล้ว

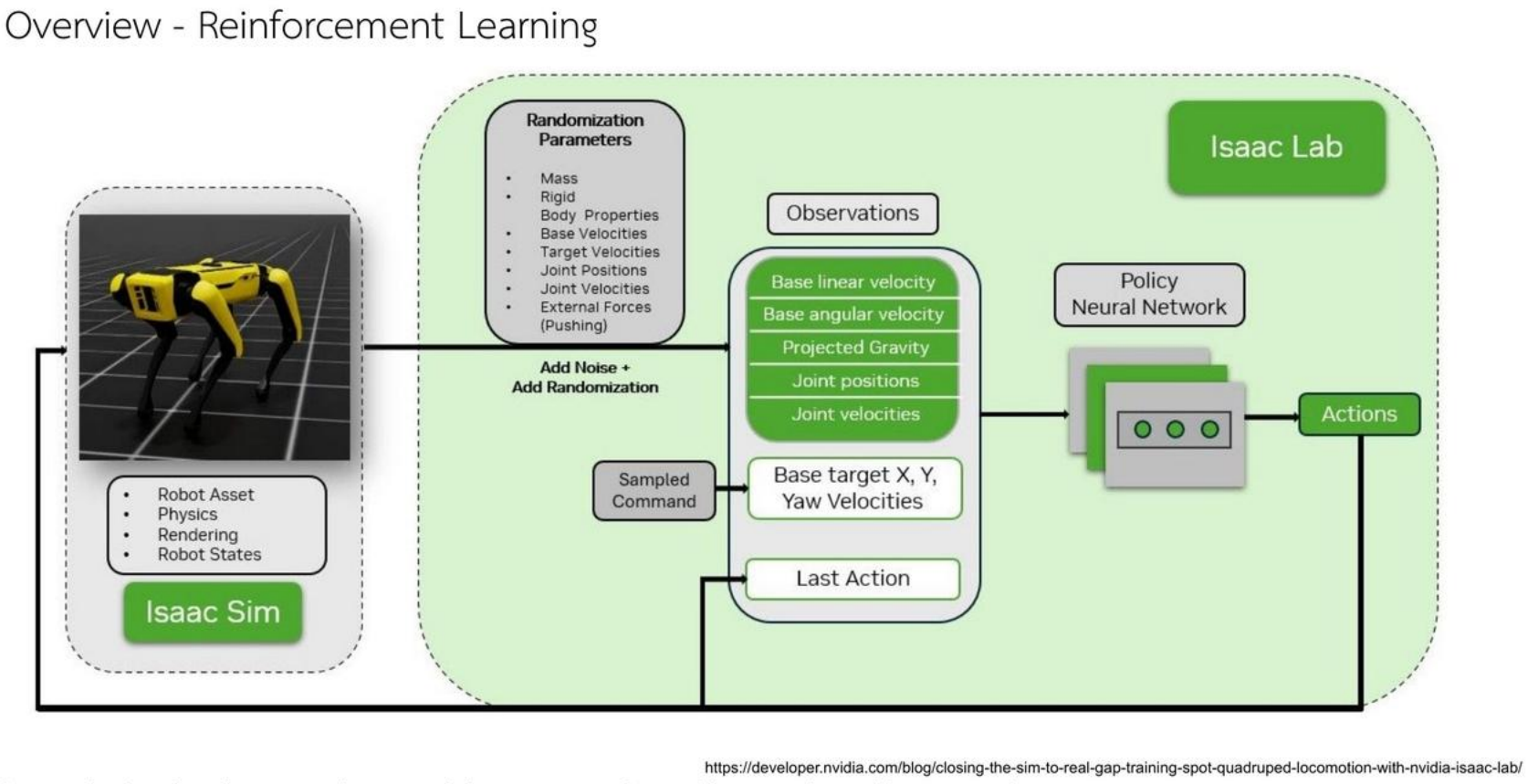
poclegs

ใส่ joint_vel และ last action ใน obs ให้ “ตอนนี้” มีข้อมูลพอ

BIG PICTURE

RL loop ใน Isaac Lab

ภาพนี้วาดเป็น Isaac Sim แต่ใน workshop เราใช้ Newton แทน

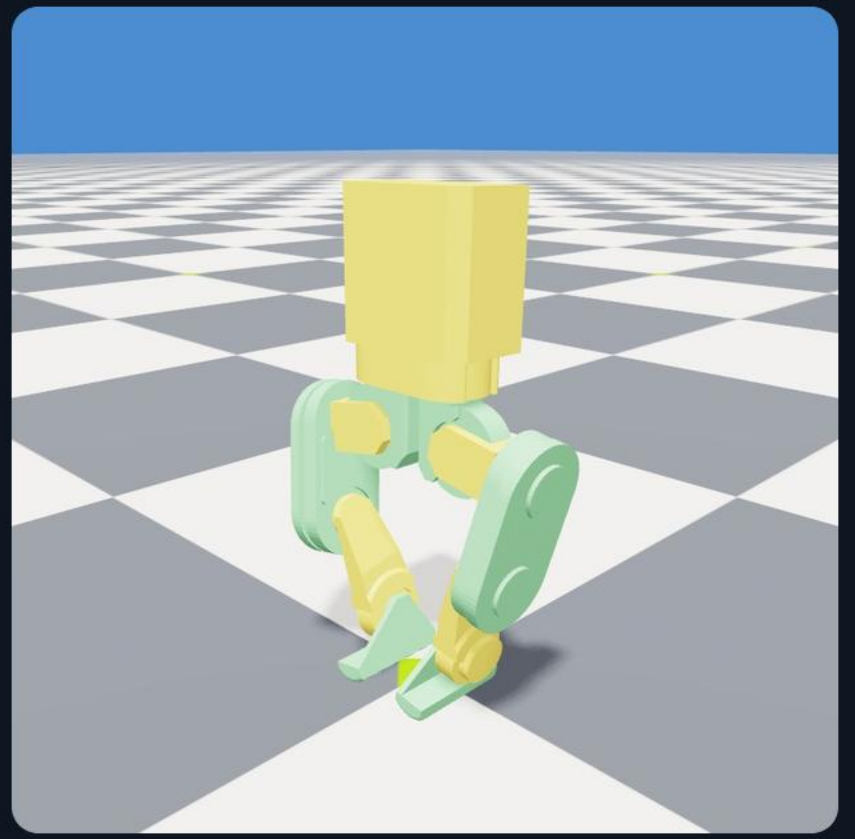


ภาพ: NVIDIA Technical Blog — Closing the sim-to-real gap: training Spot quadruped locomotion with Isaac Lab

HANDS-ON

Create own humanoid training pipeline

copy ของที่มีอยู่แล้วมาแก้ (hehe) แต่ทำบน Isaac Lab 3 + Newton



STEP 0 · PROJECT BOILERPLATE

สร้างโปรเจกต์ใหม่ด้วย isaaclab --new

```
$ cd ~/IsaacLab
$ uv run isaaclab --new --non_interactive \
    --project_path ~/projects --name poclegs_lab --author "Group X" \
    --task_name locomotion --robot_name poclegs
$ cd ~/projects/poclegs_lab && uv sync   # สร้าง .venv ของโปรเจกต์ (ไม่มี Isaac Sim)
$ SUP=~/Downloads/Simulation_for_Humanoid/Week04_RL_poclegs/week4-supplementary-file
$ PKG=~/projects/poclegs_lab/src/poclegs_lab  # ใช้ใน Step 1-3
```

เดิม (Isaac Lab 2)

```
./isaaclab.sh --new
pip install -e source/my_groupX_humanoid
python scripts/list_envs.py
```

ใหม่ (Isaac Lab 3)

```
uv run isaaclab --new
uv sync  # ไม่ต้อง pip install
uv run isaaclab list_envs
```

ไม่ใส่ --non_interactive จะถามทีละข้อ (External, Cartpole, Manager-based, rsl_rl) ทดสอบบนคอมผู้สอน 10 ต.ค. 2026

```
poclegs_lab/
├── pyproject.toml          # entry point isaaclab.tasks
├── scripts/
└── src/poclegs_lab/
    ├── assets/
    │   ├── poclegs.py      # ArticulationCfg
    │   └── data/poclegs/*.usd
    └── tasks/locomotion/
        ├── mdp/rewards.py   # reward พิเศษ
        └── config/poclegs/
            ├── __init__.py  # gym.register
            ├── env_cfg.py
            └── agents/rsl_rl_ppo_cfg.py
```

PROJECT STRUCTURE

gen ออกมาแล้วได้อะไร

assets

ไฟล์หุ่น USD + config ข้อต่อ/มอเตอร์

env_cfg + mdp

scene, observation, reward, termination

agents

ค่า PPO: ขนาด network, learning rate, iterations

ไฟล์ให้ก๊อป Week04_RL_poclegs/week4-supplementary-file
ส่วนฉบับเสร็จแล้ว Week04_RL_poclegs/poclegs_lab

STEP 0 · TEMPLATE ได้อะไรมา

gen มาเป็น Cartpole เราเปลี่ยนให้เป็น poclegs

ลบ/แก้ เพิ่ม/ค่าใหม่

template uv run isaaclab --new

```
src/poclegs_lab/
├── assets/
│   ├── __init__.py   # ASSETS_DIR
│   └── data/.gitkeep  # ว่าง
└── tasks/locomotion/
    ├── mdp/
    │   ├── __init__.pyi
    │   └── rewards.py  # joint_pos_target_l2
    └── config/poclegs/
        ├── __init__.py  # PoclegsLab-Locomotion-Poclegs
        ├── env_cfg.py   # CARTPOLE_CFG
        └── agents/rsl_rl_ppo_cfg.py  # [32,32]

$ uv run isaaclab list_envs
PoclegsLab-Locomotion-Poclegs  → LocomotionEnvCfg
(ชื่อ poclegs แต่ข้างในคือ cart-pole)
```

หลัง Step 1–3 week4-supplementary-file

```
src/poclegs_lab/
├── assets/
│   ├── __init__.py   # เหมือนเดิม
│   ├── poclegs.py    # POCLEGS_CFG    (Step 1)
│   └── data/poclegs/*.usd             (Step 1)
└── tasks/locomotion/
    ├── mdp/
    │   ├── __init__.pyi               (Step 3)
    │   └── rewards.py  # feet_air_time_range
    └── config/poclegs/
        ├── __init__.py  # 4 tasks     (Step 3)
        ├── env_cfg.py   # poclegs     (Step 3)
        └── agents/rsl_rl_ppo_cfg.py   (Step 2)

$ uv run isaaclab list_envs
Poclegs-Velocity-Flat / -Rough / -Flat-Tuned / -Flat-Urdf
```

template ให้แค่โครงไฟล์กับตัวอย่าง cart-pole ที่รันได้ เราเพิ่ม 2 อย่าง (assets/poclegs.py, data/poclegs) แก้ 5 ไฟล์ หน้าถัด ๆ ไปเทียบทีละไฟล์

TO-DO LIST

4 ขั้น จาก template ถึงหุ่นเดิน

1

Assets

USD หุ่น + ArticulationCfg

2

Agent

ค่า trainer ใน rsl_rl_ppo_cfg.py

3

Environment

env หลัก + MDP: observation, reward

4

Train

ใช้ zero agent เช็ค env ก่อนเทรน

ไฟล์ของทุกขั้นอยู่ใน **Week04_RL_poclegs/week4-supplementary-file** ก๊อปไปเติมทีละขั้น (คำสั่งครบใน README) ติดตรงไหนดูฉบับเสร็จแล้วที่ Week04_RL_poclegs/poclegs_lab

STEP 1 · ASSETS

assets/poclegs.py คือหุ่นทั้งตัวใน Python

```
POCLEGS_CFG = ArticulationCfg(
    spawn=sim_utils.UsdFileCfg(
        usd_path=f"{POCLEGS_LAB_ASSETS_DIR}/poclegs/poclegs_newton.usd",
        activate_contact_sensors=True, ...),
    init_state=ArticulationCfg.InitialStateCfg(
        pos=(0.0, 0.0, 0.449),                    # ความสูงตอนเกิด [m]
        joint_pos={"L[LR]_HAA": -0.1745, "L[LR]_HFE": -0.1745,
                   "L[LR]_KFE": 0.3491,  "L[LR]_FFE": -0.1745, ...}), # rad
    actuators={"hr": DelayedPDActuatorCfg(...), "haa": ..., "kfe": ..., "ffe": ...},
)
```

ก๊อป: `cp $SUP/assets/poclegs.py $PKG/assets/ && cp -r $SUP/assets/data/poclegs $PKG/assets/data/`

regex **L[LR]_KFE** จับได้ทั้งขาซ้ายและขวา

Isaac Lab 3: **actuator_effort_limit** / **actuator_velocity_limit** แทน effort_limit / velocity_limit

STEP 1 · ACTUATORS

มอเตอร์ 4 กลุ่ม แบบ PD + หน่วงเวลา

กลุ่ม	ข้อต่อ	kp	kd	แรงสูงสุด	delay
hr	สะโพกบิด	10	1.5	24 Nm	0–4 step
haa	สะโพกกาง	15	1.5	30 Nm	0–4 step
kfe	สะโพกพับ + เข่า	15	1.5	30 Nm	0–4 step
ffe	ข้อเท้า	10	1.5	20 Nm	0–4 step

$$\tau = kp \cdot (q^* - q) - kd \cdot \dot{q}$$

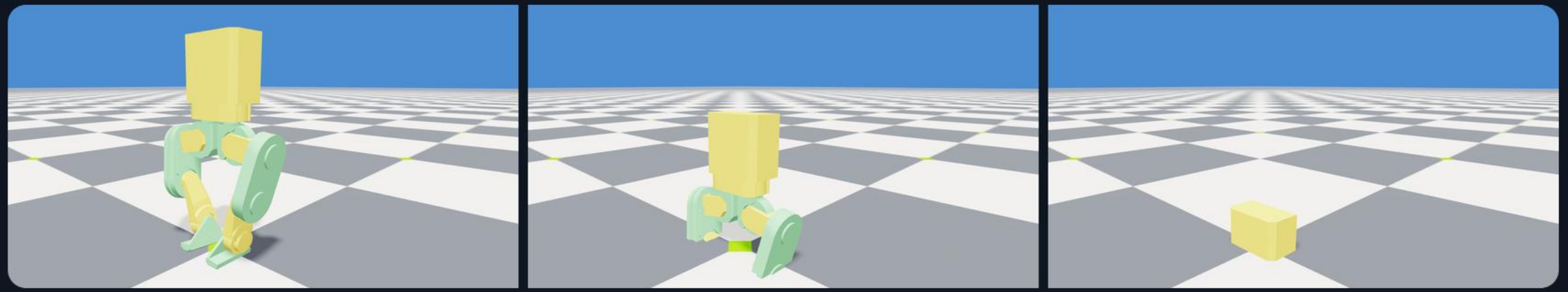
q^* = default + 0.5 × action

วัดจริง (ตรึงลำตัว): สั่ง +0.4 rad เข่าใช้ ~0.7 s ถึง 90% แต่สะโพกกางได้แค่ 0.18 rad เพราะแรงโน้มถ่วง

delay 1 step = sim.dt 5 ms หน่วงได้สูงสุด 20 ms (ค่าจากต้นฉบับ / Berkeley Humanoid)

PITFALL · เจออริงตอนเตรียม WORKSHOP

หุ่นขึ้นมาแป๊บเดียวแล้วหายไป



อาการ

ตกทะลุพื้น error No local entities ของ contact sensor

สาเหตุ

USD จาก Isaac Sim ใส่ CollisionAPI ไว้ที่ Xform แบบ instanceable Newton เลยไม่สร้าง collision shape ให้

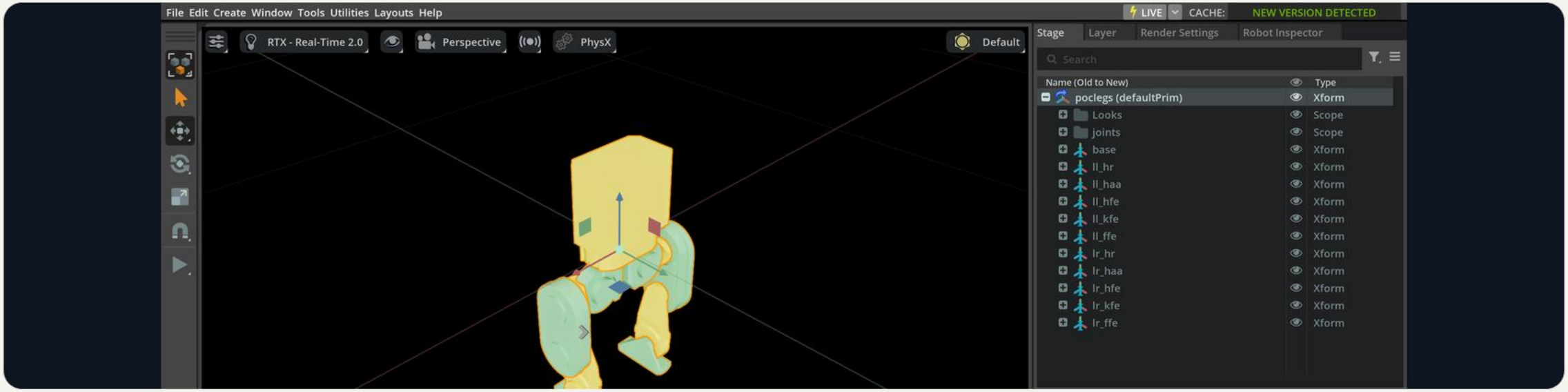
แก้

```
uv run python scripts/\
make_newton_usd.py \
poclegs.usd poclegs_newton.usd
```

STEP 1 · ดูหุ่นใน ISAAC SIM (1/3)

เปิดไฟล์ USD ของหุ่นใน Isaac Sim 6.1

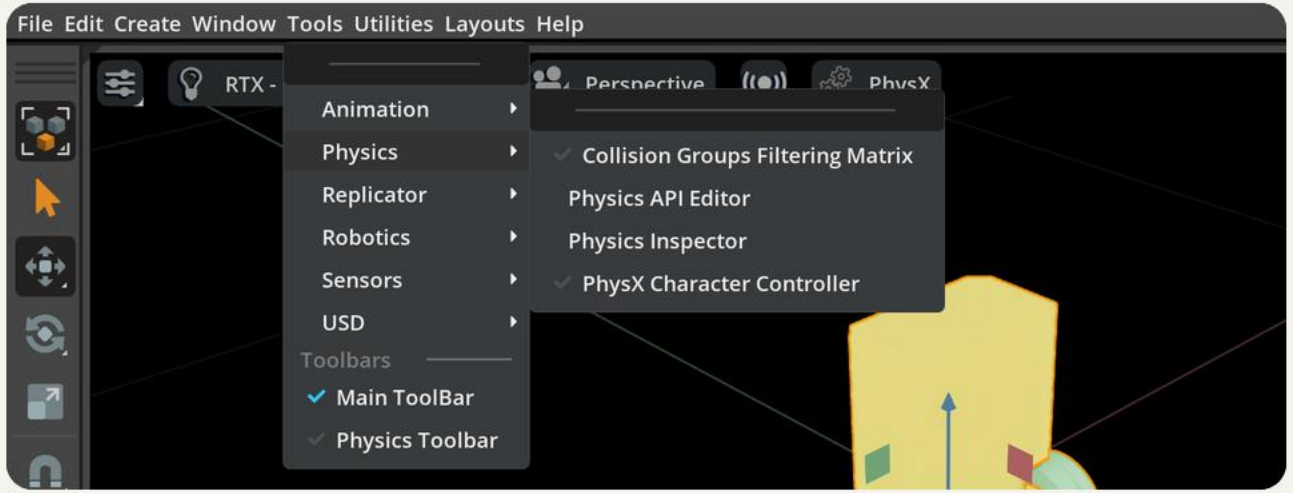
```
$ uv venv ~/isaacsim61 --python 3.12 # แยก venv ไม่แตะของ Isaac Lab
$ VIRTUAL_ENV=~/isaacsim61 uv pip install "isaacsim[all,extscache]==6.1.0.0" \
    --extra-index-url https://pypi.nvidia.com --index-strategy unsafe-best-match
$ ~/isaacsim61/bin/isaacsim # ครั้งแรกถาม EULA ตอบ Yes
```

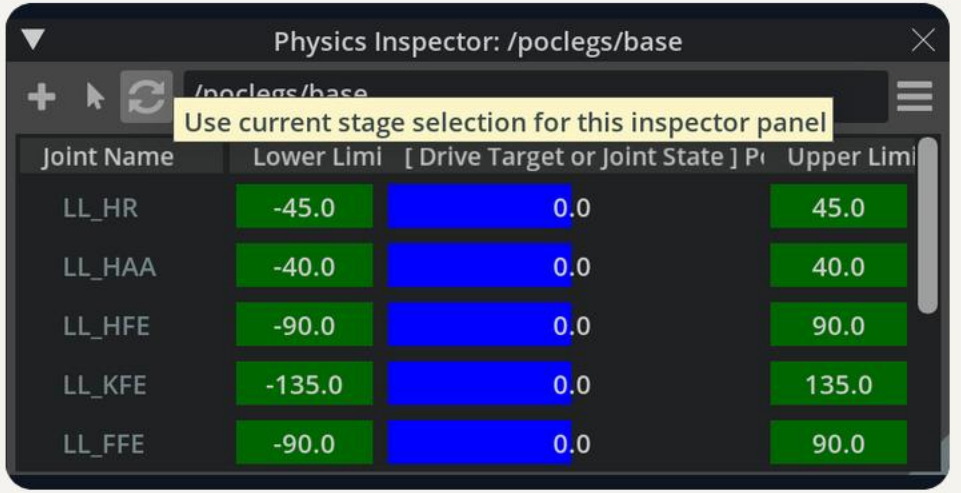


File › Open → `$PKG/assets/data/poclegs/poclegs.usd` · ใน Stage จะเห็น joints 10 ข้อ (LL_* ขาซ้าย, LR_* ขาขวา)

STEP 1 · ดูหุ่นใน ISAAC SIM (2/3)

เปิด Physics Inspector

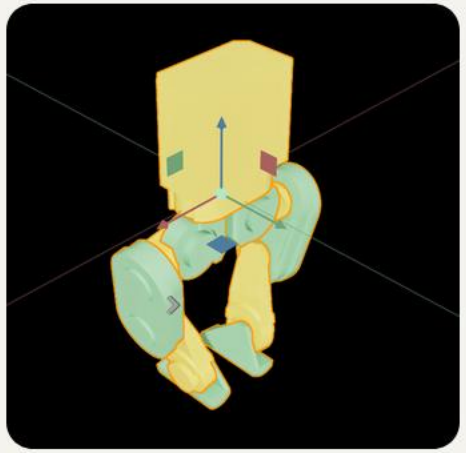




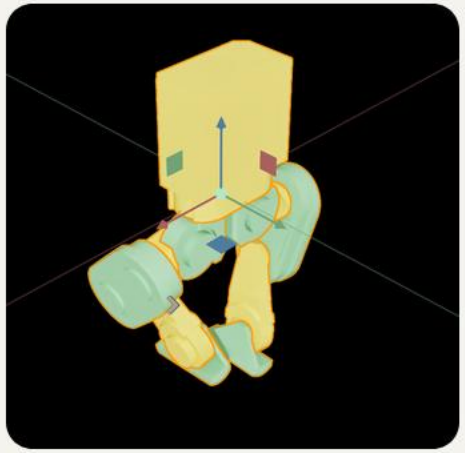
1. **Create › Physics › Physics Scene** แล้วตั้ง **Gravity Magnitude = 0** ในช่อง Property (ไม่มี scene ลากแถบแล้วหุ่นไม่ขยับ)
2. **Tools › Physics › Physics Inspector**
3. คลิก **base** ใน Stage แล้วกดปุ่ม ↻ *Use current stage selection* ในหน้าต่าง Inspector

STEP 1 · ดูหุ่นใน ISAAC SIM (3/3)

ลองเล่น: ลากแถบ = ขยับข้อต่อ (องศา)



ใส่ค่า init_state: KFE +20°



ขาซ้าย HFE −45°, KFE +90°

แถบน้ำเงิน = เป้าหมายข้อต่อ (Drive Target) ลากแล้วปล่อย หุ่นขยับตาม

เขียว ซ้าย/ขวา = limit ของข้อนั้น เกินไม่ได้

เล่นเสร็จกด ⊘ คืนค่าเดิม อย่ากด ✓ และอย่า Save ทับ USD ที่ใช้เทรน



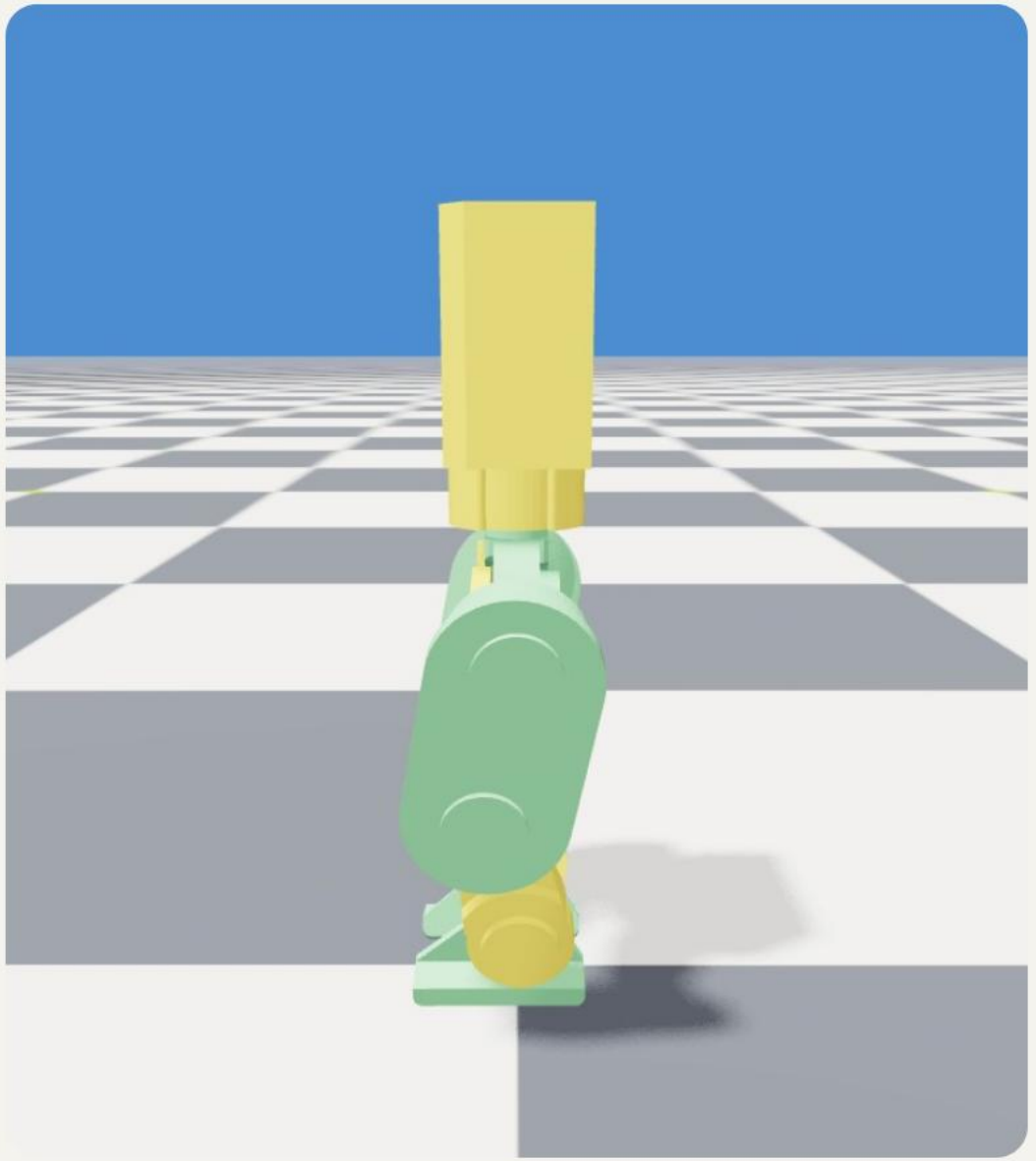
ASSIGNMENT 4.1 · INITIAL POSE

หุ่นเริ่มต้นท่าไหน?

ข้อต่อ	rad	องศา
HR	0.0	0°
HAA	−0.1745	−10°
HFE	−0.1745	−10°
KFE	+0.3491	+20°
FFE	−0.1745	−10°

โจทย์

1. แปลง init_state ใน poclegs.py เป็นองศา
2. ดูท่าจริงใน viewer ด้วย zero agent (หน้าถัดไป)
3. ใส่ค่าในตารางลง Physics Inspector ใน Isaac Sim (3 หน้าก่อน) ดูว่าท่าตรงกับรูปไหม



STEP 2 · AGENT SETTING

rsl_rl_ppo_cfg.py (rsl_rl รุ่นใหม่เปลี่ยนชื่อ)

ต้นฉบับ (เดิม)

```
empirical_normalization = False
policy = RslRlPpoActorCriticCfg(
  init_noise_std=1.0,
  actor_hidden_dims=[512,256,128],
  critic_hidden_dims=[512,256,128],
  activation="elu")
```

Isaac Lab 3

```
obs_groups = {"actor": ["policy"],
              "critic": ["policy"]}
actor = RslRlMLPModelCfg(
  hidden_dims=[512,256,128],
  activation="elu",
  distribution_cfg=...GaussianDistributionCfg(
    init_std=1.0))
critic = RslRlMLPModelCfg(hidden_dims=[512,256,128])
```

แก้ ไฟล์ที่ template gen มาให้เหมือน $SUP/agents/rsl_rl_ppo_cfg.py (ชื่อ class + experiment_name) หรือก๊อปทับ:

```
cp $SUP/agents/rsl_rl_ppo_cfg.py $PKG/tasks/locomotion/config/poclegs/agents/
```

algorithm (PPO) ใช้ค่าเดิมทั้งหมด: clip 0.2, lr 1e-3, γ 0.99, λ 0.95

experiment_name = "poclegs_flat" → logs/rsl_rl/poclegs_flat/

STEP 2 · AGENTS/RSL_RL_PPO_CFG.PY

template vs ของเรา: แก้ 7 จุด เพิ่ม class Flat

ลบ/แก้ เพิ่ม/ค่าใหม่

template (cart-pole)

```
@configclass
class PPORunnerCfg(RslRlOnPolicyRunnerCfg):
    num_steps_per_env = 16
    max_iterations = 150
    save_interval = 50
    experiment_name = "cartpole_direct"
    obs_groups = {"actor": ["policy"], "critic": ["policy"]}
    actor = RslRlMLPModelCfg(
        hidden_dims=[32, 32],
        activation="elu", ...)
    critic = RslRlMLPModelCfg(
        hidden_dims=[32, 32],
        activation="elu", ...)
    algorithm = RslRlPpoAlgorithmCfg(
        clip_param=0.2, learning_rate=1.0e-3,
        gamma=0.99, lam=0.95, ...)   # เหมือนกัน
```

ของเรา $SUP/agents/rsl_rl_ppo_cfg.py

```
@configclass
class PoclegsRoughPPORunnerCfg(RslRlOnPolicyRunnerCfg):
    num_steps_per_env = 24
    max_iterations = 3000
    save_interval = 100
    experiment_name = "poclegs_rough"
    obs_groups = {"actor": ["policy"], "critic": ["policy"]}
    actor = RslRlMLPModelCfg(
        hidden_dims=[512, 256, 128],
        activation="elu", ...)
    critic = RslRlMLPModelCfg(
        hidden_dims=[512, 256, 128],
        activation="elu", ...)
    algorithm = RslRlPpoAlgorithmCfg(
        clip_param=0.2, learning_rate=1.0e-3,
        gamma=0.99, lam=0.95, ...)   # เหมือนกัน

@configclass
class PoclegsFlatPPORunnerCfg(PoclegsRoughPPORunnerCfg):
    def __post_init__(self): ... max_iterations = 1000
                                 experiment_name = "poclegs_flat"
```

network ใหญ่ขึ้นเพราะ obs 42 ค่า / action 10 ค่า (cart-pole มีแค่ 4 / 1) ชื่อ class ต้องตรงกับตอน register (หน้า Register) และ log จะอยู่ที่ logs/rsl_rl/poclegs_flat/

STEP 3 · ENVIRONMENT CONFIG

ไม่ต้องเขียนเอง แค่ extend locomotion

```python
# เดิม: isaaclab_tasks.manager_based.locomotion.velocity.velocity_env_cfg
from isaaclab_tasks.core.velocity.velocity_env_cfg import LocomotionVelocityRoughEnvCfg

@configclass
class PoclegsRoughEnvCfg(LocomotionVelocityRoughEnvCfg):
    observations: PoclegsObservationsCfg = PoclegsObservationsCfg()
    rewards: PoclegsRewardsCfg = PoclegsRewardsCfg()

    def __post_init__(self):
        super().__post_init__()
        self.scene.robot = POCLEGS_CFG.replace(prim_path="{ENV_REGEX_NS}/Robot")
        self.scene.height_scanner = None    # poclegs ไม่มี height scan

class PoclegsFlatEnvCfg(PoclegsRoughEnvCfg): ...   # พื้นเรียบ → เทรนเร็ว
```

scene, actions, commands, events, terminations, curriculum ได้จากคลาสแม่หมดแล้ว เราแก้แค่ส่วนที่ต่าง

ก๊อปโครง: `cp $SUP/my_poclegs_env_cfg.py $PKG/tasks/locomotion/config/poclegs/env_cfg.py`
โครงนี้ Observation / Reward ยังว่าง (มี TODO) หน้าถัดไปเราเติมด้วย terms/observations.py กับ terms/rewards.py

STEP 3 · CONFIG/POCLEGS/ENV_CFG.PY

เลิกเขียนทุก manager เอง ใช้ของ locomotion

ลบ/แก้ เพิ่ม/ค่าใหม่

template 217 บรรทัด · ManagerBasedRLEnvCfg

```
from isaaclab.envs import ManagerBasedRLEnvCfg
from isaaclab_assets.robots.cartpole import CARTPOLE_CFG

class LocomotionPhysicsCfg(PresetCfg): ...
class LocomotionSceneCfg(InteractiveSceneCfg):
    robot = CARTPOLE_CFG.replace(...)
class ActionsCfg:  joint_effort = ...
class ObservationsCfg:
    joint_pos_rel, joint_vel_rel
class EventCfg:  reset_cart_position, ...
class RewardsCfg:
    alive, terminating, pole_pos, ...
class TerminationsCfg:  cart_out_of_bounds

class LocomotionEnvCfg(ManagerBasedRLEnvCfg):
    scene = LocomotionSceneCfg(num_envs=4096)
    observations / actions / events / rewards /
    terminations = ...    # ต้องเขียนเองครบ
    def __post_init__(self): dt, decimation, ...
```

ของเรา $SUP/my_poclegs_env_cfg.py (โครง) + terms/

```
from isaaclab_tasks.core.velocity.velocity_env_cfg import (
    LocomotionVelocityRoughEnvCfg, RewardsCfg)
from poclegs_lab.assets.poclegs import POCLEGS_CFG

POCLEGS_ROUGH_TERRAINS_CFG = TerrainGeneratorCfg(...)

class PoclegsObservationsCfg:   # TODO → terms/observations.py
class PoclegsRewardsCfg(RewardsCfg):  # TODO → terms/rewards.py

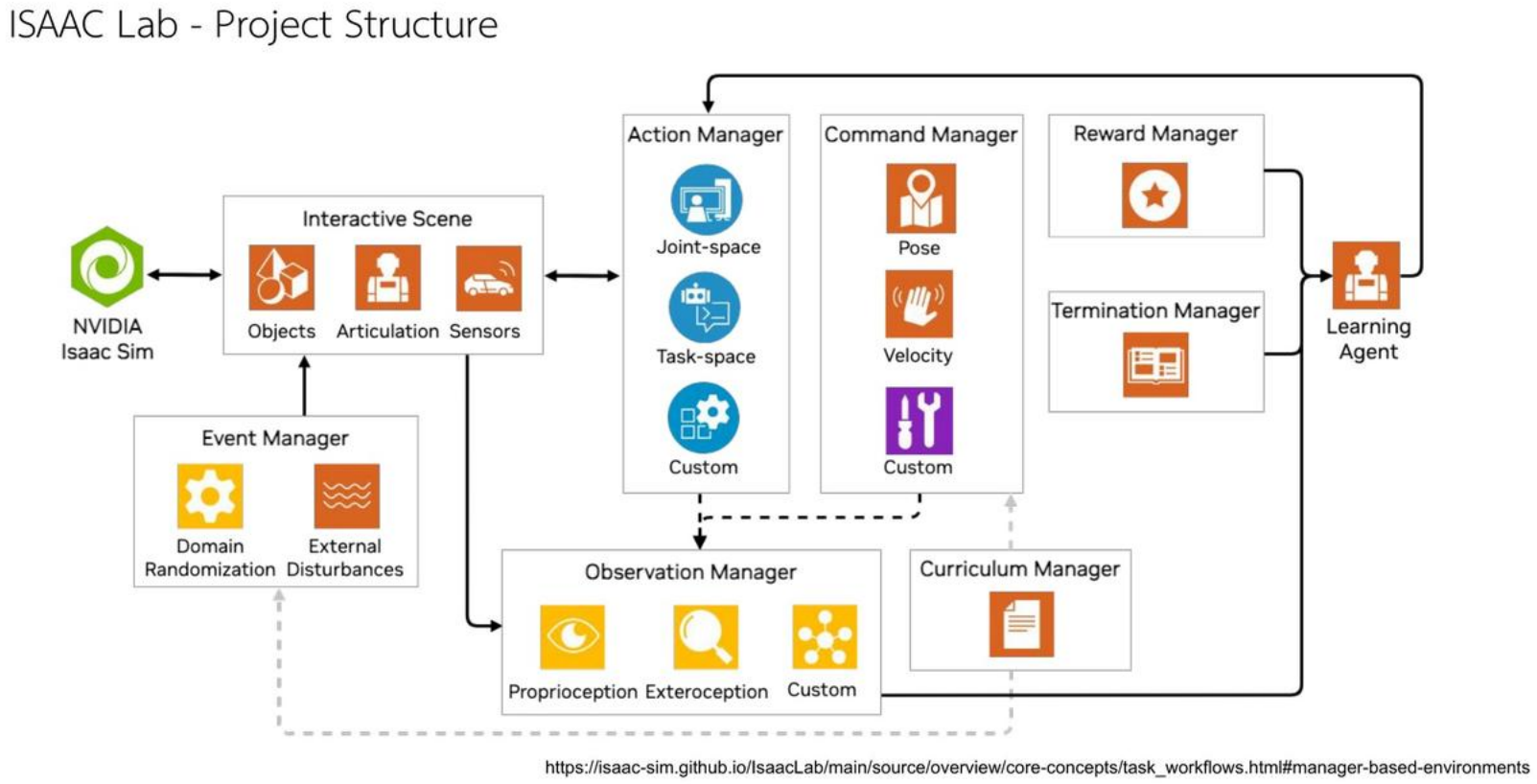
class PoclegsRoughEnvCfg(LocomotionVelocityRoughEnvCfg):
    observations = PoclegsObservationsCfg()
    rewards = PoclegsRewardsCfg()
    def __post_init__(self):
        super().__post_init__()
        self.scene.robot = POCLEGS_CFG.replace(...)
        self.scene.terrain.terrain_generator = POCLEGS_ROUGH_...
        self.scene.height_scanner = None
class PoclegsFlatEnvCfg(PoclegsRoughEnvCfg):  # พื้นเรียบ
# scene/actions/commands/events/terminations ได้จากคลาสแม่
```

ทิ้งของ cart-pole ทั้งไฟล์ (ก๊อปโครงทับ) คลาสแม่ LocomotionVelocityRoughEnvCfg มี scene, actions, commands, events, terminations ของงานเดินให้แล้ว เราเขียนแค่หุ่น, terrain, observation และ reward

MANAGER-BASED ENVIRONMENT

แต่ละ config = manager หนึ่งตัว

observations → Observation, rewards → Reward, terminations → Termination, events → Event



ภาพ: Isaac Lab docs — Task Design Workflows · Manager-based environments

STEP 3 · OBSERVATIONS

42 ค่าที่หุ่นมองเห็น

ทุก term ใส่ noise สุ่ม (Unoise) ให้ทนเซนเซอร์จริงที่ไม่แม่น

แยกมุมข้อต่อ 3 กลุ่ม เพราะ noise ต่างกัน: ข้อเท้าแรงสุด ±0.08 rad

ไม่ใช้ height_scan ตั้ง **= None**

term	dim	noise
base_lin_vel	3	±0.1 m/s
base_ang_vel	3	±0.2 rad/s
projected_gravity	3	±0.05
velocity_commands	3	—
hip_pos (HR)	2	±0.03 rad
kfe_pos (HAA, HFE, KFE)	6	±0.05 rad
ffe_pos (FFE)	2	±0.08 rad
joint_vel	10	±1.5 rad/s
actions (ครั้งก่อน)	10	—
รวม	**42**	

STEP 3 · OBSERVATIONS · อ่านโค้ด

terms/observations.py: 1 บรรทัด = 1 ค่าที่หุ่นเห็น

```
@configclass
class PoclegsObservationsCfg:
    @configclass
    class PolicyCfg(ObsGroup):  # กลุ่มที่ส่งให้ policy
        base_lin_vel = ObsTerm(func=mdp.base_lin_vel,
            noise=Unoise(n_min=-0.1, n_max=0.1))
        …
        kfe_pos = ObsTerm(func=mdp.joint_pos_rel, noise=…,
            params={"asset_cfg": SceneEntityCfg("robot",
                joint_names=[".*HAA", ".*HFE", ".*KFE"])})
        actions = ObsTerm(func=mdp.last_action)
        def __post_init__(self):
            self.enable_corruption = True
            self.concatenate_terms = True
    policy: PolicyCfg = PolicyCfg()
```

func = ฟังก์ชันที่คำนวณค่า (อยู่ใน mdp ของ Isaac Lab) เช่น ความเร็วลำตัว, มุมข้อต่อ

noise = สุ่ม ± ทุก step ตอนเทรน ให้ทนเซนเซอร์จริงได้

params = เลือกข้อต่อด้วย regex **.*HAA** ได้ทั้ง LL_HAA และ LR_HAA

ลำดับบรรทัด = ลำดับใน vector 42 ค่า, **concatenate** ต่อเป็น vector เดียว, **corruption** เปิด noise

แปะ: ใน env_cfg.py หา TODO (Step 3 · Observations) ลบ class ว่างนั้นทิ้ง แล้วแปะทั้งไฟล์ $SUP/terms/observations.py

STEP 3 · REWARDS (ค่าต้นฉบับ)

reward ของการเดินสองขา

term	weight	ต้องการให้...
track_lin_vel_xy_exp	+1.0	ความเร็ว x, y ตรงคำสั่ง
track_ang_vel_z_exp	+0.5	หมุนตัวตรงคำสั่ง
feet_air_time_range	+2.0	ยกเท้าค้าง 0.2–0.5 s ต่อก้าว (ฟังก์ชันใน mdp/rewards.py)
lin_vel_z_l2 · ang_vel_xy_l2	−2 · −0.05	ไม่เด้งขึ้นลง ไม่โยกตัว
feet_slide	−0.25	ไม่ลากเท้าไปกับพื้น
undesired_contacts	−1.0	ต้นขา/สะโพกไม่แตะพื้น
joint_deviation hip · knee	−0.1 · −0.01	ไม่บิด/กางขาเกินจำเป็น
joint_torques · action_rate	−1e-5 · −0.01	ประหยัดแรง ไม่สั่งกระตุก
flat_orientation · dof_pos_limits	0 · 0	ปิดไว้ (repo ต้นฉบับใช้ −0.5 / −1.0)

โค้ดของตารางนี้อยู่ใน **terms/rewards.py** ส่วนฟังก์ชัน feet_air_time_range อยู่ใน **mdp/rewards.py** อีก 2 หน้าจะอ่านทีละบรรทัด

STEP 3 · REWARDS · อ่านโค้ด

terms/rewards.py: ใช้ของเดิม แล้วแก้แค่ส่วนที่ต่าง

```
@configclass
class PoclegsRewardsCfg(RewardsCfg):  # ได้ track_lin_vel, action_rate … มาแล้ว
    dof_acc_l2 = None  # ปิด term ที่ไม่ใช้
    termination_penalty = RewTerm(func=mdp.is_terminated, weight=0.0)
    feet_air_time = RewTerm(
        func=mdp.feet_air_time_range,  # ฟังก์ชันของเราเอง
        weight=2.0,
        params={"sensor_cfg": SceneEntityCfg("contact_forces", body_names=".*ffe"),
            "command_name": "base_velocity",
            "threshold_min": 0.2, "threshold_max": 0.5})
    undesired_contacts = RewTerm(func=mdp.undesired_contacts, weight=-1.0,
        params={… body_names=[".*hfe", ".*haa"], "threshold": 1.0})
    …
```

RewardsCfg = reward มาตรฐานของ locomotion เราสืบทอดมาแล้วเขียนทับแค่ตัวที่อยากเปลี่ยน

= None คือปิด term, **weight 0** คือเก็บไว้แต่ไม่นับ (override เปิดทีหลังได้)

weight บวก = รางวัล, ลบ = ลงโทษ, reward ต่อ step = Σ weight × ค่า × dt

params ส่งเข้า func ตามชื่อ body_names ".*ffe" = เท้าทั้งสองข้าง

แปะ: ใน `env_cfg.py` หา `TODO (Step 3 · Rewards)` ลบ `class` ว่างนั้นทิ้ง แล้วแปะทั้งไฟล์ `$SUP/terms/rewards.py`

STEP 3 · MDP/REWARDS.PY · อ่านโค้ด

feet_air_time_range: reward ที่เราเขียนเอง

```python
def feet_air_time_range(env, command_name, sensor_cfg, threshold_min, threshold_max):
    contact_sensor = env.scene.sensors[sensor_cfg.name]
    first_contact = contact_sensor.compute_first_contact(env.step_dt).torch[:, sensor_cfg.body_ids]  # ①
    last_air_time = contact_sensor.data.last_air_time.torch[:, sensor_cfg.body_ids]  # ②
    air_time = (last_air_time - threshold_min) * first_contact  # ③
    air_time = torch.clamp(air_time, max=threshold_max - threshold_min)  # ④
    reward = torch.sum(air_time, dim=1)  # รวม 2 เท้า → 1 ค่าต่อฟุน
    reward *= torch.linalg.norm(env.command_manager.get_command(command_name)[:, :2], dim=1) > 0.1  # ⑤
    return reward  # shape (num_envs,)
```

① 1 = เท้าเพิ่งแตะพื้นใน step นี้ (จบก้าว)

② ก้าวที่เพิ่งจบลอยอยู่กี่วินาที

③ ลอย < 0.2 s ได้ค่าลบ (ลงโทษก้าวซอย)

④ ลอยเกิน 0.5 s ไม่ได้เพิ่ม (กันกระโดดค้าง)

⑤ สั่งยืนนิ่ง (|cmd| ≤ 0.1) ได้ 0

IL3: ค่าจาก sensor เป็น warp array ต้อง **.torch** ก่อนคำนวณ

ก๊อป: `cp $SUP/mdp/rewards.py $SUP/mdp/__init__.pyi $PKG/tasks/locomotion/mdp/` (__init__.pyi ทำให้เรียก mdp.feet_air_time_range ได้)

STEP 3 · TASKS/LOCOMOTION/MDP/

mdp ต้องชี้ไปที่ชุดของงานเดิน

ลบ/แก้ เพิ่ม/ค่าใหม่

template mdp/__init__.pyi + rewards.py

```
# __init__.pyi
__all__ = ["joint_pos_target_l2"]
from isaaclab.envs.mdp import *
from .rewards import joint_pos_target_l2

# rewards.py
def joint_pos_target_l2(env, target, asset_cfg):
    """Penalize joint position deviation"""
    asset = env.scene[asset_cfg.name]
    joint_pos = wrap_to_pi(asset.data.joint_pos.torch[...])
    return torch.sum(torch.square(joint_pos - target), dim=1)
```

ของเรา $SUP/mdp/

```
# __init__.pyi
__all__ = ["feet_air_time_range"]
from isaaclab_tasks.core.velocity.mdp import *
from .rewards import feet_air_time_range

# rewards.py
def feet_air_time_range(env, command_name, sensor_cfg,
                        threshold_min, threshold_max):
    first_contact = ...compute_first_contact(...).torch
    last_air_time = ...data.last_air_time.torch
    air_time = (last_air_time - threshold_min) * first_contact
    air_time = torch.clamp(air_time, max=...)
    reward = torch.sum(air_time, dim=1)
    reward *= |cmd| > 0.1 ;  return reward
```

บรรทัดสำคัญคือ **isaaclab_tasks.core.velocity.mdp** ถ้ายังเป็น isaaclab.envs.mdp ของ template จะหา mdp.feet_slide, mdp.track_lin_vel_xy_exp ไม่เจอ (AttributeError ตอนโหลด env_cfg.py)

STEP 3 · REGISTER THE TASK

ตั้งชื่อ task ให้ isaaclab เรียกได้

```
# config/poclegs/__init__.py
gym.register(
    id="Poclegs-Velocity-Flat",
    entry_point="isaaclab.envs:ManagerBasedRLEnv",
    kwargs={"env_cfg_entry_point": f"{__name__}.env_cfg:PoclegsFlatEnvCfg",
            "rsl_rl_cfg_entry_point": f"{agents.__name__}.rsl_rl_ppo_cfg:PoclegsFlatPPORunnerCfg"})
```

```
# pyproject.toml (gen ให้แล้ว)
[project.entry-points."isaaclab.tasks"]
poclegs_lab = "poclegs_lab.tasks"
```

```
$ uv run isaaclab list_envs
Poclegs-Velocity-Flat
Poclegs-Velocity-Rough
Poclegs-Velocity-Flat-Tuned (Ex 4.3)
Poclegs-Velocity-Flat-Urdf (Week 6)
```

ก๊อป: `cp $SUP/register/__init__.py $PKG/tasks/locomotion/config/poclegs/__init__.py`

STEP 3 · CONFIG/POCLEGS/__INIT__.PY

ชื่อ task + ชื่อ class ต้องตรงกันทั้ง 3 ไฟล์

ลบ/แก้ เพิ่ม/ค่าใหม่

template 1 task

```
import gymnasium as gym
from . import agents

gym.register(
    id="PoclegsLab-Locomotion-Poclegs",
    entry_point="isaaclab.envs:ManagerBasedRLEnv",
    kwargs={
        "env_cfg_entry_point":
            f"{__name__}.env_cfg:LocomotionEnvCfg",
        "rsl_rl_cfg_entry_point":
            f"{agents.__name__}.rsl_rl_ppo_cfg:PPORunnerCfg",
    })
```

ของเรา $SUP/register/__init__.py · 4 tasks

```
import gymnasium as gym
from . import agents

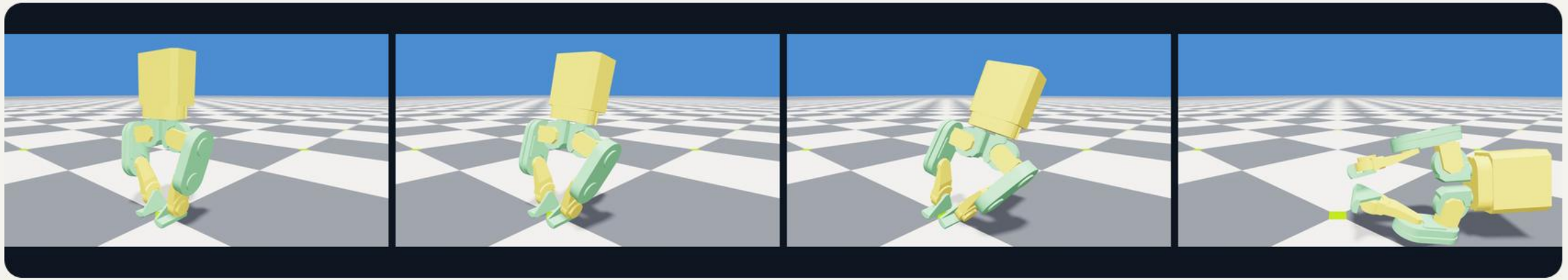
gym.register(
    id="Poclegs-Velocity-Flat",
    entry_point="isaaclab.envs:ManagerBasedRLEnv",
    kwargs={
        "env_cfg_entry_point":
            f"{__name__}.env_cfg:PoclegsFlatEnvCfg",
        "rsl_rl_cfg_entry_point":
            f"{agents.__name__}.rsl_rl_ppo_cfg:PoclegsFlatPPORunnerCfg",
    })
gym.register(id="Poclegs-Velocity-Rough", ...  PoclegsRoughEnvCfg)
gym.register(id="Poclegs-Velocity-Flat-Tuned", ... (Ex 4.3))
gym.register(id="Poclegs-Velocity-Flat-Urdf", ...  (Week 6))
```

ชื่อหลัง : ต้องมีอยู่จริง คือ PoclegsFlatEnvCfg ใน env_cfg.py และ PoclegsFlatPPORunnerCfg ใน agents/ ถ้าพิมพ์ผิดจะ error ตอน train ไม่ใช่ตอน list_envs

ASSIGNMENT 4.2 · ZERO AGENT

ส่อง env ก่อนเทรน action = 0 ตลอด

```
$ cp $SUP/scripts/*.py ~/projects/poclegs_lab/scripts/ && cd ~/projects/poclegs_lab
$ uv run isaaclab zero_agent --task Poclegs-Velocity-Flat --num_envs 16 --viz newton
```



ปกติ: ยืนท่าเริ่มต้น ล้มใน ~1 s (step 50) แล้ว reset ใหม่

ไม่ปกติ: จมพื้น ระเบิด ขาบิดแปลก ๆ ให้กลับไปเช็ค USD / init_state

ASSIGNMENT 4.3 · TRAIN

เทรน poclegs ตามค่าต้นฉบับ

```
$ uv run isaaclab train --rl_library rsl_rl --task Poclegs-Velocity-Flat \
    --num_envs 2048 --max_iterations 1500 --run_name base1500
```

24 นาที

1,500 iterations × 2,048 envs (วัดบน RTX 3060 Laptop 6 GB)

58k

steps / s ~0.9 s ต่อ iteration

30,000

iterations × 4,096 envs ในrepo ต้นฉบับ = 2.9 พันล้าน steps ≈ 14 ชม. บนโน้ตบุ๊กนี้

RESULT · ค่าต้นฉบับตรงตัว

24 นาทีผ่านไป... ยืนได้ แต่ยังไม่เดิน

13 s

ยืนนานเฉลี่ยต่อ episode (เดิม 0.3 s)

ล้ม 52% ของ episode

0.001

m/s ที่วัดได้ เมื่อสั่งเดิน 0.5 m/s

0.87

rad/s เมื่อสั่งหมุน 1.0 หมุนตัวได้

ทำไม? มาไล่หาด้วยกันใน **Exercise 4.1–4.4** ส่วนสิ่งที่แก้ไปดูหน้าถัดไป

MAKE IT WALK · POCLEGS-VELOCITY-FLAT-TUNED

3 อย่างที่เปลี่ยน

1

สั่งตามแกน y

lin_vel_y ±0.5, lin_vel_x = 0 เพราะ “หน้า” ของหุ่นคือ y

Exercise 4.1

2

ปลดเพดานความเร็ว

joint_velocity_limit = 23 / 15 / 20 / 23 rad/s แทน 1.57–3.14 จาก USD

Exercise 4.2

3

reward ชุด H1

single-stance air time, termination −200, track ใน yaw frame, flat orientation −1

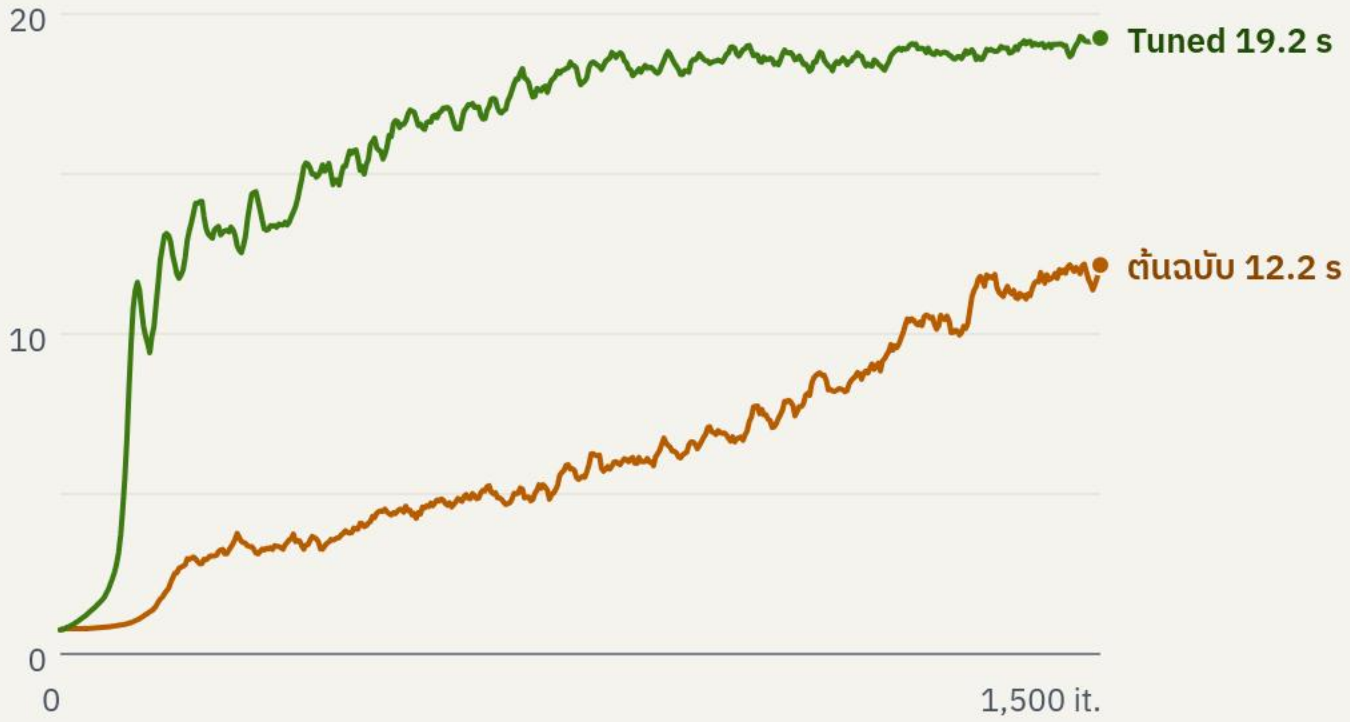
Exercise 4.3: ตัวที่สำคัญที่สุด

มอเตอร์ (kp, kd, delay) ยังใช้ค่าต้นฉบับ ทุกอย่างอยู่ใน PoclegsTunedFlatEnvCfg ใน env_cfg.py

RESULT · POCLEGS-VELOCITY-FLAT-TUNED · 1,500 IT. · 21 นาที

poclegs เดินได้แล้ว

ยืนได้นานเฉลี่ยต่อ episode [s] (เต็ม 20 s)



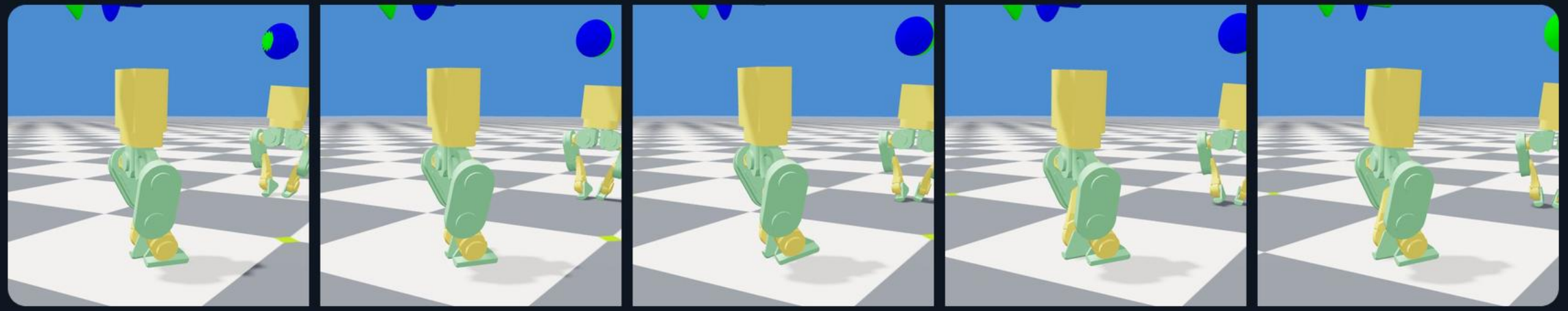
สั่ง	วัดได้	ล้ม
vy +0.4	+0.28	5
vy −0.4	−0.30	1
vy +0.2	+0.13	4
wz +0.8	+0.45	3
ยืนนิ่ง	0.00	4

eval_policy.py · 64 ตัว × 10 s · model_1499

PLAY

ดูหุ่นของเราเดินใน Newton viewer

```
$ uv run isaaclab play --rl_library rsl_rl --task Poclegs-Velocity-Flat-Tuned \
    --num_envs 16 --checkpoint latest --viz newton
# ยังเทรนไม่เสร็จ? ใช้ตัวที่แจกให้: --checkpoint pretrained/tuned1500/model_1499.pt
```



ภาพห่างกัน 0.3 s ขาสลับก้าวตามลูกศรเขียว (ทิศคำสั่ง)

ไฟล์ policy อยู่ที่ logs/rsl_rl/poclegs_flat/<run>/model_1499.pt
play ครั้งเดียวได้ exported/policy.onnx ด้วย

CHECKPOINT: ก่อนไปต่อ

Lab 4 ผ่านไหม?

4.1 · Initial pose

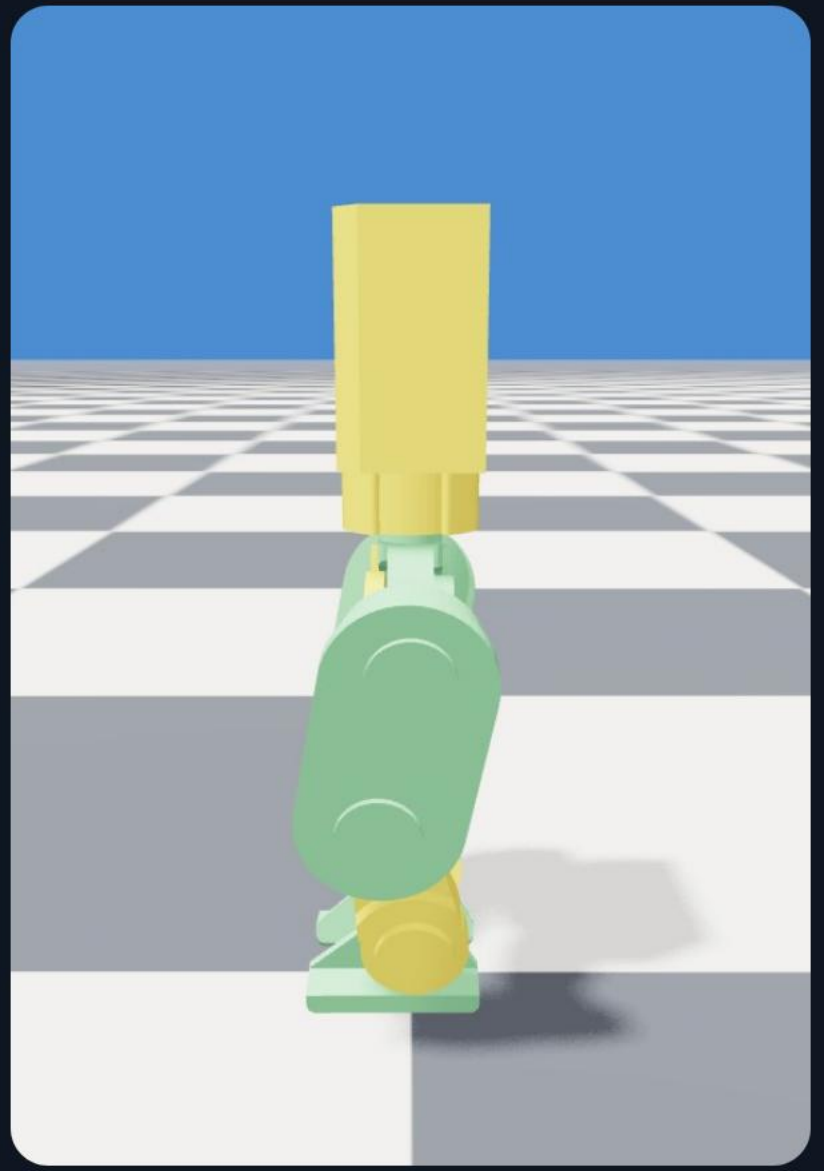
บอกท่าเริ่มต้นเป็นองศาได้ และ list_envs เห็น Poclegs-*

4.2 · Zero agent

หุ่นยืนบนพื้นแล้วล้มเองใน ~1 s ไม่จม ไม่หาย

4.3 · Train

เทรนค่าต้นฉบับจนจบ เทรน Tuned แล้ว play ให้เห็นหุ่นเดิน



EXERCISES · WEEK 4

ทำให้ poclegs เดินให้ได้

4 โจทย์ ทุกข้อเทรน 600 iterations × 2,048 envs (~9 นาที) ผลที่ลองรันจริงอยู่ในหน้าเฉลย

EXERCISE 4.1 · DIAGNOSE

หุ่นไม่เดิน? วัดก่อนเดา

4.1

```
$ EVAL_CMDS="0.5,0,0;0,0.5,0;0,0,1" uv run python scripts/eval_policy.py \
    --task Poclegs-Velocity-Flat --num_envs 64 --checkpoint latest
# แต่ละชุด = vx,vy,wz (m/s, m/s, rad/s ในกรอบลำตัว) คั่นด้วย ; · ชุดละ 10 s ทุกตัวได้คำสั่งเดียวกัน
# รายงานความเร็วเฉลี่ย (ข้าม 1 s แรก) · falls = ล้มรวมทุกตัว (ล้มแล้ว reset ล้มซ้ำได้ จึงเกิน 64 ได้)
```

ผลตอนลองรัน (BASE1500)

```
cmd vx=+0.50 → vx=+0.001  falls=18
cmd vy=+0.50 → vy=+0.000  falls=10
cmd wz=+1.00 → wz=+0.865  falls=39
```

เทรนครบ 1,500 iterations (24 นาที) หมุนตัวได้ แต่ไม่ขยับไปไหนเลย

โจทย์

1. รัน eval กับ policy ของคุณ สั่ง vx กับ vy อันไหนขยับ?
2. ดูรูปหุ่น: ขาสองข้างห่างกันตามแกนไหน พับรอบแกนไหน?
3. แล้วทิศ “หน้า” ของ poclegs คือแกนไหนของลำตัว?

คิดต่อ: Mean reward ขึ้นเรื่อย ๆ ไม่ได้แปลว่าหุ่นทำสิ่งที่เราต้องการ ต้องสั่งเองแล้ววัดว่าหุ่นทำได้จริงไหม

เฉลย · EXERCISE 4.1

“หน้า” ของ poclegs คือแกน y ไม่ใช่ x

1. ขาซ้าย/ขวาอยู่ที่ x = ±0.05 m และ HFE/KFE หมุนรอบแกน x ขาเลยแกว่งในระนาบ y–z เดินตามแกน **y**
2. คำสั่ง **lin_vel_x** ของ Isaac Lab ถือว่า x คือด้านหน้า แต่กับ poclegs มันคือ “ก้าวตะแคง” ยากมาก
3. run B (termination −200, flat_orientation −0.5, dof_pos_limits −1, คำสั่ง vx, vy ±0.5) หุ่นเริ่มขยับแต่**เฉพาะแกน y** (0.17 m/s) ส่วน vx ยังเป็น 0

run B · สั่ง	วัดได้	ล้ม
vx +0.4	+0.003	3
vy +0.4	+0.174	36
vx −0.4	−0.017	10
wz +0.8	+0.071	2

64 ตัว × 10 s · 600 iterations

```
env.rewards.termination_penalty.weight=-200.0
env.rewards.flat_orientation_l2.weight=-0.5
env.rewards.dof_pos_limits.weight=-1.0
"env.commands.base_velocity.ranges.lin_vel_x=[-0.5,0.5]"
"env.commands.base_velocity.ranges.lin_vel_y=[-0.5,0.5]"
```

เช็คแกนของ base frame ก่อนเทรนทุกครั้ง ถ้าหุ่นหันผิด ให้แก้ช่วงคำสั่ง (lin_vel_y) หรือหมุน frame ของ base

EXERCISE 4.2 · JOINT SPEED LIMIT

เข่าหมุนได้เร็วสุดเท่าไหร่?

4.2

```
# ใน script ที่สร้าง env (เช่น zero agent ของคุณ)
print(env.unwrapped.scene["robot"].data.joint_vel_limits.torch[0])
$ … train … env.scene.robot.actuators.kfe.joint_velocity_limit=20.0 …
# หน่วย rad/s · กลุ่ม kfe = สะโพกพับ (HFE) + เข่า (KFE) · ใช้กับ Poclegs-Velocity-Flat (Tuned ตั้งไว้ 23/15/20/23 แล้ว)
```

ผลตอนลองรัน

```
joint_vel_limits [rad/s]
HR 3.14 · HAA 1.57 · HFE 3.14
KFE 1.57 · FFE 3.14
```

มาจาก maxJointVelocity ใน USD (90°/s, 180°/s) ไม่ใช่ actuator_velocity_limit = 15–23 ใน config

โจทย์

1. ก้าวขา 0.5 s เข่าต้องหมุนไปกลับ ~0.6 rad ต้องเร็วกี่ rad/s?
2. ตั้ง **joint_velocity_limit** ทั้ง 4 กลุ่มเท่า actuator แล้วเทรนซ้ำ
3. หุ่นเดินได้หรือยัง?

คิดต่อ: Isaac Lab 3 แยก **actuator_velocity_limit** (โมเดลมอเตอร์) กับ **joint_velocity_limit** (เพดานของ solver) ออกจากกัน

เฉลย · EXERCISE 4.2

เพดานความเร็วต้องแก้ แต่ยังไม่พอ

1. 0.6 rad × 2 ใน 0.5 s ≈ **2.4 rad/s** เฉลี่ย (ช่วงพีคสูงกว่านี้อีก) เพดาน 1.57 rad/s ของเข่าจึงต่ำเกินไป
2. run D = run B + joint_velocity_limit 23/15/20/23 ep. len แทบเท่าเดิม (936 vs 915 step) ล้ม 24% vs 19% ของ episode
3. ยังไม่เดิน: vy +0.023 เมื่อสั่ง 0.4 policy ติดที่ “ยืนนิ่งก็ได้ reward พอแล้ว” ต้องแก้ reward (4.3)

run D · สั่ง	วัดได้	ล้ม
vx +0.4	+0.002	7
vy +0.4	+0.023	15
vy −0.4	−0.001	11
wz +0.8	+0.086	26

64 ตัว × 10 s · 600 iterations

ค่าใน USD (จาก URDF/CAD) มีผลกับฟิสิกส์จริง ได้หุ่นตัวใหม่เมื่อไหร่ ให้อ่าน data.joint_vel_limits / joint_effort_limits ก่อน

EXERCISE 4.3 · REWARD RECIPE

ยืม reward ของ H1 มาใช้

4.3

```
$ uv run isaaclab train --rl_library rsl_rl --task Poclegs-Velocity-Flat-Tuned \
    --num_envs 2048 --max_iterations 600 --run_name tuned600
# env_cfg.py: PoclegsTunedRewardsCfg — ดัดแปลงจาก core/velocity/config/h1/rough_env_cfg.py
# โปรเจกต์ที่สร้างเอง: แปะ $SUP/terms/tuned.py ต่อท้าย env_cfg.py ก่อน (ตรง TODO Exercise 4.3)
```

term	ต้นฉบับ	Tuned (H1)
air time	range 0.2–0.5 s ×2.0	single stance ×1.0
termination	—	−200
track xy / yaw	1.0 / 0.5	1.0 / 1.0 (yaw frame)
flat orientation	0	−1.0
คำสั่ง	vx, vy ±1	vy ±0.5, vx 0

โจทย์

1. เทรน Tuned 600 iterations แล้ว eval ด้วยคำสั่ง vy ±0.4
2. term ไหนน่าจะทำให้ต่างที่สุด? ปิดทีละตัวด้วย **env.rewards.<ชื่อ term>.weight=0.0** เช่น termination_penalty (ดูชื่อในตาราง Reward Manager ตอนเริ่มเทรน)

คิดต่อ: feet_air_time แบบต้นฉบับ **ลงโทษ**ก้าวที่สั้นกว่า 0.2 s หุ่นที่ยังก้าวไม่เป็นเลยเลยเลือก “ไม่ก้าว” ไว้ก่อน

เฉลย · EXERCISE 4.3

reward แบบ H1 ทำให้หุ่นเริ่มก้าวเดิน

1. run E = reward ต้นฉบับ + termination −200 + flat −0.5 + dof_pos −1, คำสั่ง vy ±0.6 (vx ±0.2), joint limit เหมือน Tuned ได้ vy แค่ 0.05–0.09
2. Tuned (run F): เดินได้ทั้งสองทิศ 0.17–0.18 m/s และหมุนตัว 0.23 rad/s ในเวลาเทรนเท่ากัน (9 นาที)
3. E มี termination −200 อยู่แล้ว ตัวที่ต่างจริงคือ air time แบบ single stance (ให้รางวัลการยืนขาเดียว), track แบบ yaw frame (yaw ×1.0) และ flat orientation −1

สั่ง	E · ต้นฉบับ	F · Tuned
vy +0.4	+0.047	+0.182
vy −0.4	−0.088	−0.169
wz +0.8	+0.061	+0.230
ล้ม (vy +0.4)	32	15

64 ตัว × 10 s · 600 iterations ทั้งคู่

เริ่มจาก reward ที่ใช้ได้ผลกับหุ่นคล้าย ๆ กันใน Isaac Lab แล้วค่อยปรับ เร็วกว่าคิดเองจากศูนย์

EXERCISE 4.4 · ACTUATORS

มอเตอร์แข็งขึ้น / ไม่มี delay

4.4

```
# (ก) kp ×3, kd ×2 บน task Tuned — ต้องใส่ครบ 4 กลุ่ม (hr, haa, kfe, ffe)
… --task Poclegs-Velocity-Flat-Tuned env.scene.robot.actuators.hr.stiffness=30
env.scene.robot.actuators.haa.stiffness=45
  env.scene.robot.actuators.kfe.stiffness=45 env.scene.robot.actuators.ffe.stiffness=30  # + damping=3.0 ทั้ง 4 กลุ่ม
# stiffness = kp (Nm/rad) · damping = kd (Nm·s/rad) · ค่าเดิม kp 10/15/15/10, kd 1.5
# (ข) ปิด delay บน task เดิม (ต้นฉบับ): max_delay นับเป็น physics step (1 step = 5 ms)
… --task Poclegs-Velocity-Flat env.scene.robot.actuators.kfe.max_delay=0 …  # ทั้ง 4 กลุ่ม · เดิมสุ่ม 0–4 step (0–20 ms) ใหม่ทุก
reset
```

ทำไมน่าลอง

ยืนขาเดียว สะโพกกางต้องรับ ~14 Nm (17.5 kg × g × ~8 cm) kp 15 Nm/rad ต้องคลาดจากเป้าเกือบ 1 rad ถึงจะออกแรงได้เท่านั้น

โจทย์

1. (ก) เดินเร็วขึ้นไหม? ล้มมากขึ้นหรือน้อยลง?
2. (ข) delay 0–20 ms ทำให้เทรนยากขึ้นแค่ไหน?
3. หุ่นจริงควรตั้งค่าแบบไหน?

คิดต่อ: ค่าที่ทำให้ sim เทรนง่ายที่สุด อาจไม่ตรงกับมอเตอร์จริง sim-to-real ต้องใช้ค่าที่วัดจากของจริง

เฉลย · EXERCISE 4.4

มอเตอร์แข็งเร็วขึ้นแต่ล้มง่าย มี delay เทรนช้า

1. (ก) kp ×3: vy +0.30 m/s (Tuned 0.18) หมุน 0.43 rad/s (0.23) แต่ล้ม 12–37 ครั้ง มากกว่าแทบทุกคำสั่ง
2. (ข) ไม่มี delay (reward ต้นฉบับ): iteration 600 ยืนได้ 466 step vs 216 step ตอนมี delay เทรนเร็วขึ้นราว 2 เท่า แต่**ยังไม่เดิน**
3. หุ่นจริงมี delay และ gain จำกัด เทรนแบบมี delay เอาไปใช้จริงปลอดภัยกว่า แม้ใน sim จะช้ากว่า

สั่ง	Tuned	kp ×3
vy +0.4	+0.18 · 15	+0.30 · 12
vy −0.4	−0.17 · 13	−0.23 · 32
wz +0.8	+0.23 · 14	+0.43 · 37
ยืนนิ่ง	0 · 9	0 · 20

ความเร็วที่วัดได้ · จำนวนครั้งที่ล้ม (64 ตัว × 10 s, 600 it.)

ปรับ actuator ให้ sim เทรนง่ายขึ้นได้ แต่ต้องรู้ว่ากำลังเปลี่ยนหุ่น ค่าที่ใช้กับหุ่นจริงต้องวัดจากมอเตอร์จริง